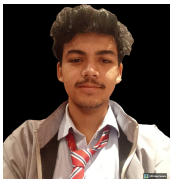
PROJECT AND TEAM INFORMATION

Project Title

JalRakshak: Intelligent Water Resource Allocation and Scheduling System

Team Name: *Team #*	*HydroLab* *OSDBMS-V-2026-T111*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	 *Gupta, Aditya – 240211284* *eddygupta1513@gmail.com*
Team member 2 *(Last Name, name: student ID: email, picture):*	 *Rashi – 240221622* *240221622@geu.ac.in*
Team member 3 *(Last Name, name: student ID: email, picture):*	 *Chaudhary, Ritesh – 240211067* *240211067@geu.ac.in*
Team member 4 *(Last Name, name: student ID: email, picture):*	 *Rawat, Ritesh – 24021276* *24021276@geu.ac.in*

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

The problem
Water is limited and many users need it at the same time like hospitals homes schools and factories
There is not enough for everyone so someone must decide who gets how much
Many places serve the first person who asks and this is not fair because a hospital may wait while a small shop gets water

Our idea
We treat water like a computer resource and each water request like a process that waits for it
This lets us use simple ideas from Operating Systems and Databases to share water in a fair and safe way

What we built
A C++ scheduler decides who gets how much water
A PostgreSQL database keeps every decision safe using locks and transactions so the same water is never given twice
A Python FastAPI backend joins the database and the scheduler and a Docker setup starts everything with one command

Our test case
The reservoir has 50000 litres and the users ask for 80000 litres so there is a real shortage to handle

Progress and next steps
Phase 1 built the full pipeline with the FCFS rule which serves requests in the order they come
Phase 2 added the Priority Round Robin and Water-Aware rules which are fairer
Next we will compare all the rules and add demand prediction and a dashboard for officials

Limits and goal
It is a decision tool and does not control real pumps
The project supports SDG 6 which is clean water and sanitation

Updated Project Approach and Architecture (2 pts)

Design: PostgreSQL stores nine linked tables for zones users reservoirs requests runs allocations consumption predictions and alerts. A C++ scheduler runs four rules. A Python FastAPI backend connects all parts. A React dashboard comes later.

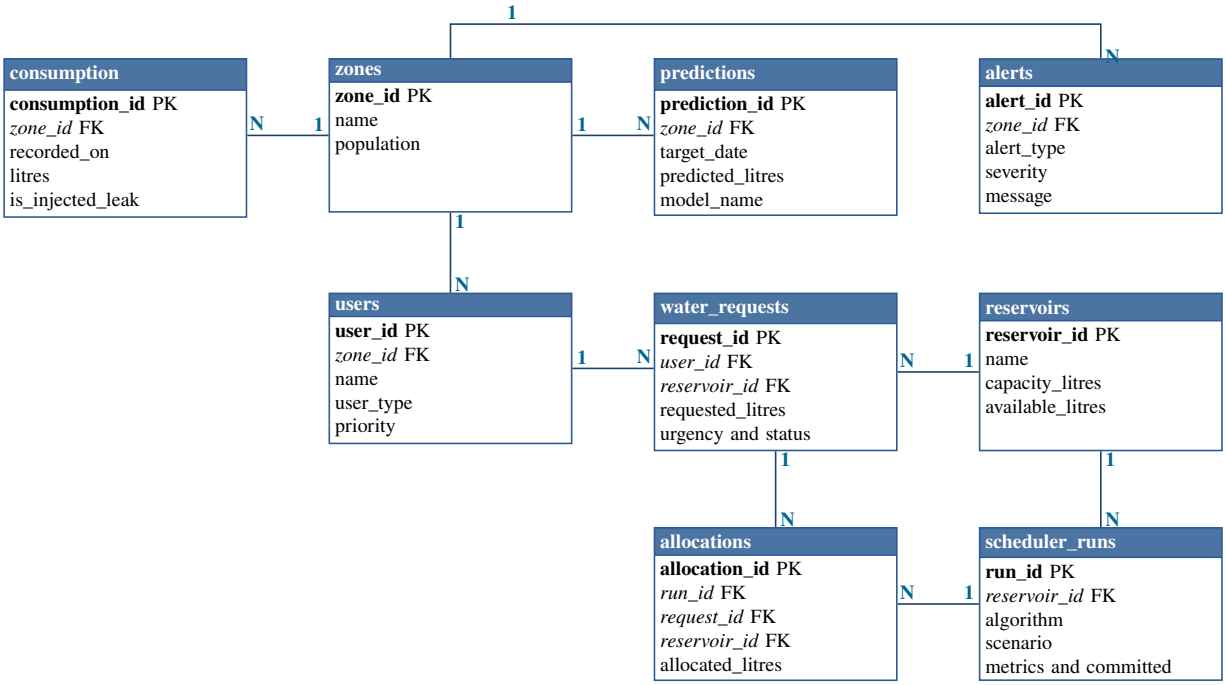
The four rules: FCFS serves requests in the order they came. Priority serves the most important users first like a hospital. Round Robin gives each request a fixed number of litres in every turn so nobody is left out. Water-Aware gives each request a score from priority urgency water scarcity people affected and fairness. The weights of the score are kept in a setting file.

Communication: REST API for the tester and JSON between the backend and the scheduler.

Flow: the backend reads pending requests and locks the reservoir. It sends the data to the scheduler and gets a plan with six measures. It saves the run the allocations the new request status and the new water level in one transaction. If one step fails nothing is saved.

Tools: C++ Make Python FastAPI PostgreSQL Docker Compose and React

ER diagram: the nine tables from schema.sql and how they link (PK is the main key and FK is the link to another table and 1 and N show one to many)



Tasks Completed (7 pts)

Task Completed	Team Member
Database with nine linked tables with rules and sample data of 8 requests that total 80000 L against a 50000 L reservoir	Ritesh Rawat
C++ scheduler with the FCFS rule and six measures and 48 automatic checks	Rashi
Priority rule that serves the most important users first and Round Robin rule that gives each request a fixed share in turns	Rashi
Water-Aware scheduler that gives each request a score and uses weights	Aditya Gupta
FastAPI backend with nine endpoints and a safe commit that locks the reservoir and saves in one transaction	Ritesh Chaudhary
10 automatic Python tests for the backend	Aditya Gupta
Docker Compose setup that starts everything with one command and the files contracts.md and README.md	Aditya Gupta & Ritesh Chaudhary

Challenges/Roadblocks (7 pts)

FCFS is unfair. It serves by arrival and not by need. The hospital waited 54 minutes and a school and a colony were rejected only because they came late. We fixed this with the Priority Round Robin and Water-Aware rules.

Same water given twice. Two requests can grab the last water at the same time. We solved it with a reservoir lock one transaction and a database rule that blocks a negative level. A many user demo is still to do.

Choosing weights. The Water-Aware score needs good weights. We keep them in a setting file and test many values to find a good balance.

Joining C++ and Python. The two parts must understand each other. We solved it with one JSON format written in contracts.md.

Checking new rules. We check each rule with hand made answers so we know the result is right.

No real data. We use made up data and the design allows real data later.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
Concurrency demo that sends many requests at once with and without locks	Ritesh Rawat
Data science module for demand prediction and leak alerts	Ritesh Chaudhary & Aditya Gupta

Project Outcome/Deliverables (2 pts)

A working prototype that shares limited water among many users and shows which rule is fairest.

Done: a PostgreSQL database for water data requests and history. A C++ scheduler with the FCFS Priority Round Robin and Water-Aware rules. Safe allocation with locks and transactions. A FastAPI backend. A Docker setup that starts the system with one command.

To come: a comparison of the rules on normal medium and severe shortage with tables and charts. A Python module for demand prediction and leak alerts. A React dashboard for officials. Full documents and a demo

Progress Overview (2 pts)

Done: the full base is ready. A request can travel from the database through the C++ scheduler and the API and back into the database. All four rules work which are FCFS Priority Round Robin and Water-Aware. The safe all or nothing transaction and the Docker setup are also done and this lowers risk for later work.

Behind: we have not yet compared the four rules on normal medium and severe shortage. The many user demo with and without locks is also not done. These are the next tasks.

Not started: the data science module and the dashboard. The dashboard waits on purpose because it is best to build it when there are results to show.

Codebase Information (2 pts)

Repository link: https://github.com/AdityaGupta-codes/JalRakshak
Branch: main
Important commits: 5 commits on the main branch which add the project folder jalrakshak and the rule files fcfs and priority queue and round robin

Languages: C++ for the scheduler then Python for the backend then SQL for the database

Main folders: jalrakshak for the project code. In the design it holds db for schema.sql and seed.sql then scheduler for C++ code tests and Makefile then backend for the FastAPI app Dockerfile and Python tests then docs for contracts.md then experiments for sample input and output files

Main files: fcfs and priority queue and round robin for the three rules then docker-compose.yml and README.md

How to run: run docker compose up in the project folder then open localhost:8000/docs to test every endpoint

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
C++ unit tests	Pass	48 checks including the 50000 L example
New rule tests	Pass	Priority Round Robin and Water-Aware checked
Water limit check	Pass	total water given never goes above water available
Wrong input check	Pass	bad input is refused with a clear message
Python API tests	Pass	10 tests including lock first and rollback on failure
Database load	Pass	schema and sample data load
Docker start	Pass	one command starts both services
Concurrency test	Pending	many requests with and without locks

Deliverables Progress (2 pts)

PostgreSQL database: **completed**
FastAPI backend: **completed**
C++ scheduler with four rules: **completed**
Safe allocation with locks and transactions: **completed** and the many user demo is pending
Docker setup: **completed**
Comparison of the rules: **pending**
Python data science module: **pending**
React dashboard: **pending**
Testing: **in progress**
Documents: **in progress** and contracts.md and README.md are done